G18 Project 1: xv6 System Call Implementation

Group 18

Contents

1 Project Overview

2 Objectives

3 Analysis of Existing System Calls

4 Implementation of New System Calls

5 Build and Run
- 5.1 Build xv6
- 5.2 Run User Program

6 Project Structure

7 System Call Design

8 Completed Functionalities
- 8.1 Process Tracking
- 8.2 Inter-Process Communication (IPC)
- 8.3 Shared Memory
- 8.4 Thread Support
- 8.5 Signal Handling
- 8.6 Locks (Synchronization)
- 8.7 Scheduling

9 Error Handling Approach

10 Contribution of Group Members

11 Conclusion

1 Project Overview

This project involves modifying the xv6 operating system to implement new and enhanced system calls.

The goal is to extend kernel functionalities such as process creation, inter-process communication (IPC), threading, synchronization, and signal handling.

User-level programs are added to demonstrate the working of these system calls.

2 Objectives

- Analyze existing system call implementation in xv6
- Modify and implement new system calls
- Implement functionalities like IPC, threads, locks, and signals
- Demonstrate usage using a user program
- Maintain clean and modular kernel code

3 Analysis of Existing System Calls

In xv6, system calls provide an interface between user programs and the kernel.

Each system call follows a structured flow:

- User program invokes a syscall
- Control switches to kernel mode
- Syscall handler executes the required functionality

Important files involved:

- `syscall.c` – syscall dispatcher
- `syscall.h` – syscall numbers
- `sysproc.c` – syscall implementations
- `usys.pl` – user-level bindings

4 Implementation of New System Calls

New system calls were added and existing ones were modified to support enhanced functionalities.

Steps followed:

- Added syscall numbers in `syscall.h`
- Registered syscalls in `syscall.c`
- Implemented functions in kernel files

- Added user-level wrappers in `user.h`
- Updated `usys.pl`

5 Build and Run

5.1 Build xv6

```
make clean
make qemu
```

5.2 Run User Program

```
test_syscalls
```

6 Project Structure

```
xv6-project/
 kernel/
 user/
 include/
 Makefile
 screenshots/
```

7 System Call Design

System calls follow this execution flow:

$$\text{User Program} \rightarrow \text{System Call Interface} \rightarrow \text{Kernel} \rightarrow \text{Execution}$$

Steps to add a system call:

- Define syscall number
- Add entry in dispatcher
- Implement kernel function
- Add user wrapper

8 Completed Functionalities

The following system calls were implemented and integrated into the xv6 kernel:

8.1 Process Tracking

System Call: `getppid`

Purpose: Returns the parent process ID of the calling process.

Usage:

```
int parent = getppid();
```

Description: This system call helps in tracking process hierarchy.

Output of getppid:

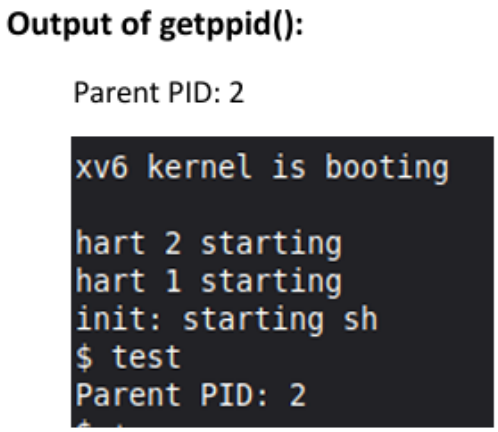

Figure 1: Output of getppid system call

8.2 Inter-Process Communication (IPC)

System Calls: `send`, `recv`

Purpose: Enable communication between processes using message passing.

Usage:

```
send(pid, msg);
recv(buffer);
```

Description: Processes communicate by sending and receiving messages through kernel-managed buffers.

Output of IPC:

Output of IPC:

Value sent: 100

Value received: 100

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ ipctest
Value sent: 100
Value received: 100
```

Figure 2: IPC using send and recv

8.3 Shared Memory

System Call: `shmget`

Purpose: Provides a shared memory region that can be accessed by multiple processes.

Usage:

```
int *ptr = shmget();
*ptr = 1234;
```

Description: Shared memory is implemented in the kernel and mapped into the address space of processes. It allows direct data sharing without copying, making it faster than message passing.

Demonstration: The parent process writes to shared memory, the child reads and modifies it, and the parent verifies the updated value.

Output:

```
parent wrote: 1234
child read: 1234
child wrote: 5678
parent read after child: 5678
shared memory test passed!
```

Output Screenshot:

```
$ shmtest
shmget: segment id = 0
parent wrote: 1234
child read:  1234
child wrote: 5678
parent read after child: 5678
shared memory test passed!
$
```

Figure 3: Shared memory communication between parent and child

8.4 Thread Support

System Call: `clone`

Purpose: Provides kernel-level support for threads.

Usage:

```
clone(function, arg);
```

Description: Threads share the same address space but execute independently, enabling concurrency.

Output of Threads:

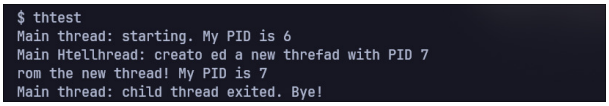

Figure 4: Thread creation using clone

8.5 Signal Handling

System Calls: `alarm_signal`, `sigreturn`

Purpose: Handle asynchronous events and process notifications.

Usage:

```
alarm_signal(ticks);
sigreturn();
```

Description: Signals notify processes about events, and `sigreturn` restores the process state after handling.

Output of Signals:

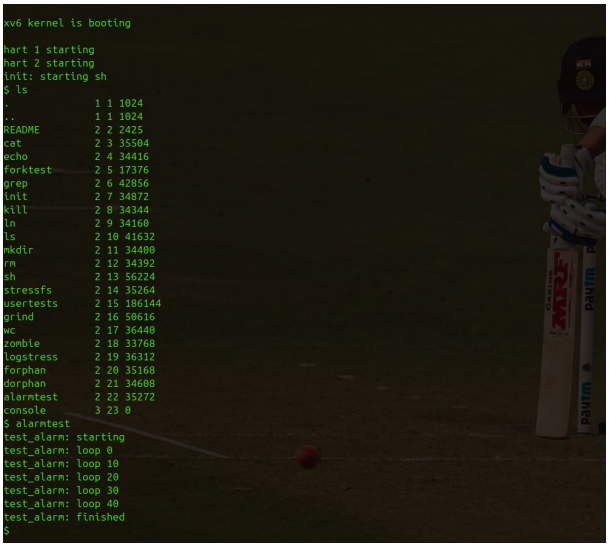

Figure 5: Signal handling demonstration

8.6 Locks (Synchronization)

System Calls: `mylock_init`, `mylock_acquire`, `mylock_release`, `increment`, `getcounter`

Purpose: Ensure safe access to shared resources and prevent race conditions in concurrent processes.

Usage:

```
mylock_init();

mylock_acquire();
increment();          // critical section
mylock_release();

int val = getcounter();
```

Description: A custom lock mechanism is implemented in the kernel using a shared lock variable. The lock acquisition uses an atomic instruction (`__sync_lock_test_and_set`) to ensure mutual exclusion.

A shared kernel variable `shared_counter` is used as a critical resource.

- **increment():** Increases the value of `shared_counter`
- **getcounter():** Returns the current value of `shared_counter`

Demonstration: Two child processes are created using `fork()`. Each process increments the shared counter 1000 times.

Without locks, a race condition would lead to incorrect results. With locks, only one process accesses the critical section at a time, ensuring correctness.

Output:

```
Final counter: 2000
```

Output of Locks:

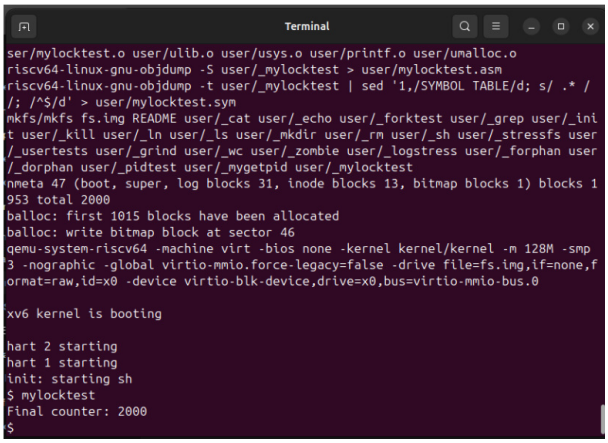

Figure 6: Synchronization using custom lock system calls

8.7 Scheduling

System Calls: `yield`, `setpriority`, `getpriority`

Purpose: Implement priority-based process scheduling.

Usage:

```
setpriority(pid, priority);
getpriority(pid);
yield();
```

Description: Processes can yield CPU voluntarily and adjust priorities to influence scheduling decisions.

Output of Scheduling:

Figure 7: Priority-based scheduling output

9 Error Handling Approach

The system ensures robustness by handling invalid inputs, unsafe operations, and kernel-level failures across all implemented system calls.

- Invalid system call arguments (e.g., negative PID, null pointers, or out-of-range values) return appropriate error codes such as `-1`.
- Attempting to send a message using `send()` to a non-existent process ID results in failure.
- `recv()` fails gracefully if no message is available in the process buffer.
- Shared memory operations (`shmget`) handle errors when memory allocation fails or when the shared region is already at capacity.
- In thread creation using `clone()`, invalid function pointers or stack misconfiguration may cause process termination; such cases are validated before execution.
- Lock system calls (`mylock_acquire`) prevent deadlock scenarios by checking lock ownership and ensuring atomic access using test-and-set operations.
- Repeated unlocking using `mylock_release()` without ownership results in ignored operations or error flags.
- Signal handling (`alarm_signal`, `sigreturn`) ensures that invalid signal states or missing handlers do not crash the process and restore safe execution context.
- Scheduling system calls (`setpriority`, `getpriority`) validate priority ranges and reject invalid values outside allowed limits.
- System protects against invalid process references during scheduling or IPC operations (e.g., accessing already terminated processes).

- Kernel debug messages are printed for failed operations to assist in tracing and debugging system behavior.

10 Contribution of Group Members

- **24JE0689 - Satyavarapu Surya Sitha Harini Sreeja**
  - Set up the xv6 project baseline.
  - Implemented `getppid` and IPC system calls (`send`, `recv`) for inter-process communication.
- **24JE0690 - Saurabh Prajapat**
  - Implemented the `clone` system call for lightweight process creation.
  - Validated thread execution using `thtest`.
- **24JE0691 - Shaik Zaheer Mushtaq**
  - Implemented shared memory system calls (`shmget`, `shmat`, `shmdt`).
  - Enabled efficient cross-process data sharing.
- **24JE0692 - Shalvi Kumari**
  - Implemented user-accessible lock system calls (`mylock_init`, `mylock_acquire`, `mylock_release`).
  - Handled synchronization functionality for critical section protection.
- **24JE0693 - Shivam Chaudhary**
  - Implemented signal handling system calls (`alarm_signal`, `sigreturn`).
  - Integrated kernel-side timers with user-space signal handlers.
- **24JE0694 - Shlok Radadia**
  - Implemented scheduling system calls (`setpriority`, `getpriority`, `yield`).
  - Integrated dynamic priority-based scheduling into xv6 kernel.

11 Conclusion

This project successfully extends the xv6 operating system by implementing new system calls.

It provides practical exposure to kernel-level programming, process management, synchronization, and inter-process communication.